```
"""
================================================================================
JAGAPRIYAN B
CH.SC.U4CSE24216
LAB EXERCISE 9: K-NEAREST NEIGHBOURS (KNN)

Topics Covered:
  Exercise 1: Handwritten Specimen / Digit Recognition using KNN (Machine Learning in Action)
  Exercise 2: Color Classification (Brightness, Saturation) with Euclidean Distance Calculation
================================================================================
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sn
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_digits
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler

print("="*70)
print("EXERCISE 9.1: HANDWRITTEN SPECIMEN RECOGNITION USING KNN")
print("="*70)
# Optical Handwritten digits dataset (same format as pbharris / ML in action handwriting dataset)
digits = load_digits()

print(f"Total number of digit images: {digits.data.shape[0]}")
print(f"Features per image (8x8 pixel intensities): {digits.data.shape[1]}")
print(f"Classes: {np.unique(digits.target)}")

# Display the first 4 sample digits
fig, axes = plt.subplots(1, 4, figsize=(8, 3))
for ax, image, label in zip(axes, digits.images[:4], digits.target[:4]):
    ax.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')
    ax.set_title(f'Label: {label}')
    ax.axis('off')
plt.suptitle("Sample Handwritten Specimens")
plt.show()

# Split dataset
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(
    digits.data, digits.target, test_size=0.25, random_state=42
)

# Test multiple K values as demonstrated in lab manual
k_values = [1, 3, 5, 7, 9]
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
    knn.fit(X_train_d, y_train_d)
    y_pred = knn.predict(X_test_d)
    acc = accuracy_score(y_test_d, y_pred)
    accuracies.append(acc)
    print(f"k = {k} | Accuracy = {acc:.4f} ({acc*100:.2f}%)")

# Best Model Evaluation (k=3)
best_knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
best_knn.fit(X_train_d, y_train_d)
y_pred_best = best_knn.predict(X_test_d)

print(f"\nFinal Accuracy on Test Set (k=3): {accuracy_score(y_test_d, y_pred_best):.4f}")
print("\nClassification Report (Handwritten Digits):")
print(classification_report(y_test_d, y_pred_best))

# Confusion Matrix
plt.figure(figsize=(7, 6))
cm_digits = confusion_matrix(y_test_d, y_pred_best)
sn.heatmap(cm_digits, annot=True, fmt='d', cmap='Blues')
plt.title("KNN Handwritten Digits Confusion Matrix")
plt.xlabel("Predicted Digit")
plt.ylabel("Actual Digit")
plt.show()


print("\n" + "="*70)
print("EXERCISE 9.2: EUCLIDEAN DISTANCE & COLOR CLASSIFICATION")
print("="*70)
# Exact Table from Lab Manual (Page 43):
```

```
# Exact Table from Lab Manual (Page 43):
# Brightness | Saturation | Class
# 40         | 20         | Red
# 50         | 50         | Blue
# 60         | 90         | Blue
# 10         | 25         | Red
# 70         | 70         | Blue
# 60         | 10         | Red
# 25         | 80         | Blue

color_data = {
    'Brightness': [40, 50, 60, 10, 70, 60, 25],
    'Saturation': [20, 50, 90, 25, 70, 10, 80],
    'Class': ['Red', 'Blue', 'Blue', 'Red', 'Blue', 'Red', 'Blue']
}

df_color = pd.DataFrame(color_data)
print("Dataset provided in Lab Manual (Page 43):")
print(df_color)

# Test new query point: Brightness = 30, Saturation = 40
query_point = np.array([30, 40])
print(f"\nNew entry to classify: Brightness = {query_point[0]}, Saturation = {query_point[1]}")

# 1. Step-by-step Euclidean Distance Formula:
# Distance = sqrt( (x1 - x2)^2 + (y1 - y2)^2 )
distances = []
for idx, row in df_color.iterrows():
    pt = np.array([row['Brightness'], row['Saturation']])
    dist = np.sqrt(np.sum((pt - query_point)**2))
    distances.append(dist)

df_color['Euclidean_Distance_to_Query'] = np.round(distances, 2)
df_sorted = df_color.sort_values(by='Euclidean_Distance_to_Query')

print("\nDistances from the new entry (sorted by proximity):")
print(df_sorted[['Brightness', 'Saturation', 'Class', 'Euclidean_Distance_to_Query']])

# 2. KNN Classifier with k=3
X_color = df_color[['Brightness', 'Saturation']].values
y_color = df_color['Class'].values

knn_color = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
knn_color.fit(X_color, y_color)

predicted_color = knn_color.predict([query_point])
probabilities = knn_color.predict_proba([query_point])

print(f"\nKNN (k=3) Predicted Class for point [30, 40]: -> {predicted_color[0]}")
for cls, prob in zip(knn_color.classes_, probabilities[0]):
    print(f"Probability of {cls}: {prob*100:.1f}%")

# Visualization: Scatter plot of training points + Query point
plt.figure(figsize=(7, 5))
for cls, color in [('Red', 'red'), ('Blue', 'blue')]:
    subset = df_color[df_color['Class'] == cls]
    plt.scatter(subset['Brightness'], subset['Saturation'], color=color, s=90, label=f'Class: {cls}')

plt.scatter(query_point[0], query_point[1], color='green', marker='*', s=250, label='New Entry [30, 40]')
plt.title("KNN Color Classification (Brightness vs Saturation)")
plt.xlabel("Brightness")
plt.ylabel("Saturation")
plt.legend()
plt.grid(True)
plt.show()
```